Capstone Project

Integrity Risk Assessment Agent

The **Integrity Risk Assessment Agent** is a reasoning-driven component within a unified integrity platform that evaluates whether a company is safe to partner with. It consolidates signals from disparate sources financial indicators, regulatory actions, sanctions lists, ESG controversies, reputational signals, and operational disruptions and produces fast, evidence-based assessments for risk and compliance professionals, procurement teams, legal departments, and business stakeholders. This design embeds a **Tree-of-Thought (ToT) memory update and conflict-resolution module** into the agent's RAG and multi-agent workflow so the system can robustly interpret, disambiguate, and persist new evidence about third parties, especially under cold-start conditions where the vector database has little or no historical memory for a newly encountered entity.

Where ToT reasoning improves the agent and why linear CoT is insufficient

The most fragile part of the Integrity Risk Assessment workflow is **entity interpretation and memory update**: when the agent ingests a new feed item, news article, or user report about a company, it must decide whether the observation updates an existing entity profile, creates a new entity, represents a temporal change, or is an erroneous or low-confidence signal. Under cold-start, retrieval returns sparse or no vectors, increasing ambiguity and the number of plausible interpretations.

A linear chain-of-thought (CoT) tends to **prematurely commit** to a single interpretation and propagate that decision into the memory store. In practice this leads to incorrect canonicalization (e.g., overwriting a correct historical fact), conflation of distinct entities with similar names, or loss of alternative hypotheses that later evidence would have resolved. ToT is required because it explicitly **enumerates parallel hypotheses**, evaluates them against constraints and evidence, and preserves runner-ups as low-confidence alternatives. This prevents early noise from becoming permanent and supports conservative, auditable updates that are essential for compliance workflows.

ToT structure and what a thought represents

A **thought** is a compact reasoning state representing a **candidate memory update hypothesis**. Each thought encodes:

- **Interpretation** of the new input (correction, temporal change, new attribute, duplicate entity, or new entity).

- **Proposed memory actions** (attribute set, confidence assignment, timestamping, entity split/merge).
- **Assumptions and constraints** (e.g., "assumes same legal entity," "assumes source reliability X," "cold-start: preserve alternatives").
- **Local score vector** capturing consistency, plausibility, and cost.

A **node** is a single thought. A **branch** is a sequence of nodes forming a reasoning path from raw input to a complete update plan. **Depth** is the number of reasoning steps; typical depths are **3–5** (interpret → retrieve/compare → propose updates → validate → finalize). The **branching factor** at the initial interpretation is expected to be **3–6** (new attribute, correction, temporal update, alternative hypothesis, different entity, defer). A hard depth cap of **5–6** bounds latency and compute.

Cold-start is first-class: when retrieval yields low density or no vectors, the thought generator explicitly produces conservative branches that preserve ambiguity (store alternatives with low confidence), avoid destructive overwrites, and include hypotheses that defer canonicalization until corroborating evidence arrives.

How reasoning is represented as a tree and termination criteria

The ToT search constructs a tree where each path ends in a **complete update plan**: a set of memory writes, confidence adjustments, timestamps, and optional alternative hypotheses to persist. The controller runs a **beam search** (primary strategy) that maintains a fixed number of top branches at each depth. Termination occurs when:

- Surviving branches reach a complete update plan, or
- Depth or compute limits are reached, in which case the controller selects the best available branch and optionally persists runner-ups as non-canonical alternatives.

Final selection favors the highest composite score but preserves alternatives under cold-start. The system records the chosen branch's justification and the set of preserved hypotheses for auditability.

Evaluation and pruning mechanism

Each branch is scored by a composite rubric:

- **Consistency** with high-confidence facts and schema constraints.
- **Cold-start conservatism**: penalty for over-commitment when memory is sparse; reward for preserving alternatives.

- **Temporal coherence**: plausibility of change over time and timestamp alignment.
- **Disambiguation quality**: evidence for same vs different entity.
- **Source reliability and user intent alignment.**
- **Operational cost**: estimated DB and compute cost for applying the update.

Evaluation is hybrid: a **critic agent** (LLM-based) performs semantic scoring and justification; **heuristic checks** enforce hard constraints (type mismatches, timestamp paradoxes); **tool calls** (vector DB similarity, cluster density, provenance checks) supply objective signals. The critic writes per-branch metadata into shared state for the controller.

Pruning rules remove branches that violate hard constraints or fall below a score threshold. If all branches are pruned, the system executes a **conservative fallback**: persist the new observation as a low-confidence, non-canonical fact and mark the entity ambiguous for later resolution. **Tie resolution** uses reranking favoring cold-start preservation and, if needed, a small LLM ensemble vote.

Search strategy selection and compute constraints

Beam search is chosen as the primary strategy because it balances exploration of multiple plausible interpretations with bounded compute. Beam width (e.g., **3–5**) controls breadth; depth caps control latency. BFS would be too expensive; DFS risks premature commitment; Monte Carlo sampling is less deterministic for auditable memory writes. Heavy tool calls (vector DB, provenance checks) are reserved for top beam candidates to reduce cost. The design enforces strict beam and depth caps and selective tool invocation to meet latency and budget constraints typical in enterprise risk workflows.

Mapping ToT roles to implementation tools and insertion point

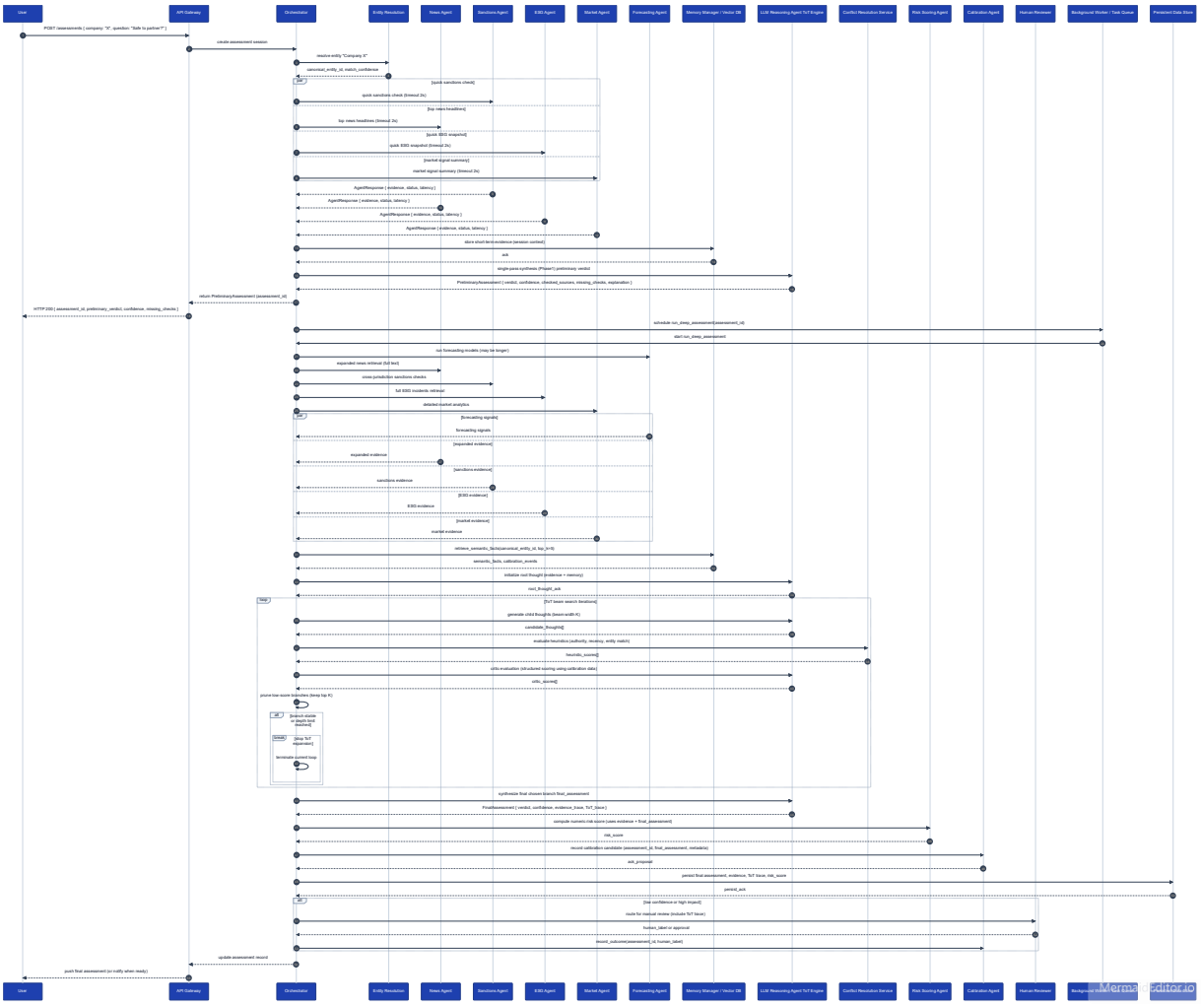
- **Thought generator:** CrewAI agent role for hypothesis generation; LangChain for prompt templates and tool wrappers; MCP for branch state.
- **Critic/evaluator:** CrewAI critic agent for semantic scoring; LangChain evaluation chains for heuristics and tool integration; MCP stores per-branch scores.
- **Decision maker/controller:** CrewAI controller orchestrates beam search; LangChain implements control flow; MCP maintains active beam and global state.
- **Memory/state manager:** MCP is authoritative for branch and global state; LangChain tools interface with the vector DB and structured store for retrieval and writes.

The ToT module is inserted between **retrieval** and **memory write**: after initial parsing and retrieval (which may be sparse under cold-start), ToT generates candidate update plans, evaluates them, and returns a selected update plus optional preserved alternatives to the memory manager.

Risk and mitigation

Risk: Branch explosion and weak evaluation under cold-start leading to high compute and poor decisions.
Mitigation: Enforce strict beam width and depth caps; use a cold-start-aware scoring function that rewards conservative, non-destructive updates; limit heavy tool calls to top beam candidates; implement a fallback that stores ambiguous observations as low-confidence alternatives until corroborating evidence arrives.

Sequence Diagram, end-to-end flow



***Note**, I used Microsoft Copilot to review the project and design ideas. All project concepts, and final decisions are my own.*